

Transformers

Tabla de contenidos

1	Una arquitectura basada en atención	3
1.1	Antes del Transformer	3
1.2	La idea fundamental	4
1.3	Información posicional	4
1.4	Bloques Transformer	5
1.5	Atención multicabeza	6
1.6	Redes feed-forward	6
1.7	Conexiones residuales	7
1.8	Normalización	7
1.9	Encoder y decoder	7
1.9.1	Encoder	8
1.9.2	Decoder	8
1.10	Variantes de la arquitectura	8
1.10.1	Modelos de tipo encoder	9
1.10.2	Modelos de tipo decoder	9
1.10.3	Modelos encoder-decoder	9
1.11	Procesamiento en paralelo	9
1.12	Capas y representaciones	10
1.13	Transformer y ventana de contexto	10
1.14	El Transformer no es un LLM	11
1.15	Una arquitectura dentro de un sistema inteligente	12
1.16	Del Transformer al modelo de lenguaje	12

1 Una arquitectura basada en atención

En las secciones anteriores hemos presentado las piezas necesarias para representar y relacionar el lenguaje:

- los **tokens** dividen el texto en unidades procesables;
- los **embeddings** convierten esas unidades en representaciones numéricas;
- la **atención** permite relacionar cada token con los demás elementos del contexto.

El **Transformer** reúne estas capacidades dentro de una arquitectura completa para procesar secuencias.

Fue presentado en 2017 por Vaswani y sus colaboradores en el artículo *Attention Is All You Need* (Vaswani et al. 2017). Su propuesta consistía en utilizar la atención como mecanismo central del modelo, sin depender de redes recurrentes o convolucionales para procesar la secuencia.

Esta arquitectura constituye la base de gran parte de los modelos de lenguaje modernos.

1.1. Antes del Transformer

El lenguaje es una secuencia ordenada.

Para interpretar una frase, el modelo debe considerar:

- qué elementos aparecen;
- en qué orden;
- qué relaciones existen entre ellos;
- qué información anterior resulta relevante;
- cómo cambia el significado según el contexto.

Antes de la aparición del Transformer, muchas arquitecturas procesaban las secuencias de forma principalmente progresiva.

Un elemento se analizaba después del anterior, manteniendo un estado que intentaba conservar la información relevante:

```
token 1 → token 2 → token 3 → token 4
```

Este enfoque permitía trabajar con secuencias, pero presentaba algunas dificultades:

- limitaba la paralelización del entrenamiento;

- complicaba el tratamiento de relaciones muy distantes;
- obligaba a transportar información a través de numerosos pasos;
- podía perder detalles importantes en secuencias largas.

El Transformer propone una organización diferente.

En lugar de recorrer necesariamente el texto token a token para relacionar sus elementos, permite comparar directamente múltiples posiciones mediante autoatención.

1.2. La idea fundamental

En un Transformer, cada token construye su representación teniendo en cuenta los demás tokens disponibles en el contexto.

De forma simplificada:

```
tokens
  ↓
embeddings e información posicional
  ↓
autoatención
  ↓
transformaciones neuronales
  ↓
representaciones contextuales
```

Estas operaciones se organizan en bloques que se repiten varias veces.

Cada bloque recibe las representaciones producidas por el bloque anterior, establece nuevas relaciones y genera representaciones progresivamente más elaboradas.

Por tanto, el Transformer no aplica una única operación de atención.

Utiliza múltiples cabezas de atención, transformaciones neuronales y numerosas capas sucesivas.

1.3. Información posicional

La atención permite relacionar tokens, pero necesita también conocer su posición dentro de la secuencia.

Consideremos:

El perro persigue al gato.

y:

El gato persigue al perro.

Ambas frases contienen prácticamente los mismos tokens, pero su orden modifica completamente el significado.

Por ello, el Transformer añade información posicional a los embeddings de entrada.

De forma conceptual:

```
embedding del token + información de posición
      ↓
representación de entrada
```

Esta información permite distinguir:

- qué token aparece antes;
- cuál aparece después;
- la distancia entre posiciones;
- el orden general de la secuencia.

La propuesta original utilizaba codificaciones posicionales basadas en funciones matemáticas (Vaswani et al. 2017). Arquitecturas posteriores han desarrollado otros mecanismos para representar la posición y las distancias relativas entre tokens.

1.4. Bloques Transformer

Un Transformer se construye mediante la repetición de bloques.

Aunque existen distintas variantes, un bloque suele incluir dos componentes principales:

1. un mecanismo de atención;
2. una red neuronal que transforma individualmente la representación de cada posición.

De forma simplificada:

```
representaciones de entrada
      ↓
    atención multicabeza
      ↓
red neuronal feed-forward
      ↓
representaciones de salida
```

A estos componentes se añaden conexiones residuales y mecanismos de normalización que facilitan el entrenamiento de redes profundas.

1.5. Atención multicabeza

La atención multicabeza permite calcular varias relaciones en paralelo.

Cada cabeza genera sus propias consultas, claves y valores, y puede aprender a identificar patrones diferentes.

Una cabeza podría resultar sensible a determinadas relaciones sintácticas; otra, a referencias entre elementos; otra, a asociaciones semánticas o temporales.

No se asigna manualmente una función fija a cada cabeza. Estas especializaciones aparecen durante el entrenamiento y no siempre pueden interpretarse con claridad.

El proceso puede representarse así:

	cabeza 1	
representaciones	cabeza 2	combinación → resultado
	cabeza 3	
	cabeza n	

El resultado de todas las cabezas se combina y se transforma para obtener una nueva representación de cada token.

1.6. Redes feed-forward

Después de la atención, cada posición atraviesa una pequeña red neuronal denominada normalmente **feed-forward**.

La atención combina información entre diferentes posiciones.

La red feed-forward transforma la información resultante de cada posición.

Podemos distinguir ambas funciones:

- la **atención** determina qué información debe relacionarse;
- la **red feed-forward** transforma esa información.

Estas redes utilizan los mismos parámetros para todas las posiciones de la secuencia, aunque procesan una representación contextual diferente en cada una.

1.7. Conexiones residuales

Los bloques Transformer incorporan conexiones residuales.

En lugar de sustituir completamente la entrada de una capa por el nuevo resultado, parte de la información original se suma a la transformación realizada.

De forma simplificada:

```
entrada
  ↓
transformación
  ↓
resultado + entrada ←
```

Estas conexiones ayudan a:

- conservar información;
- entrenar arquitecturas con muchas capas;
- facilitar el flujo de la señal y del error;
- reducir algunos problemas asociados a redes muy profundas.

1.8. Normalización

El Transformer también utiliza mecanismos de normalización para mantener valores internos en rangos adecuados y estabilizar el entrenamiento.

La combinación habitual de:

- atención;
- redes feed-forward;
- conexiones residuales;
- normalización;

forma la unidad básica que se repite a lo largo de la arquitectura.

Cada nueva capa recibe las representaciones producidas por la anterior y las refina.

1.9. Encoder y decoder

La arquitectura presentada originalmente estaba formada por dos grandes componentes:

- **encoder;**
- **decoder.**

1.9.1. Encoder

El encoder recibe una secuencia de entrada y construye representaciones contextuales de sus elementos.

Cada posición puede relacionarse con todas las demás posiciones de la entrada.

Por ejemplo, en un sistema de traducción, el encoder podría procesar la frase escrita en el idioma de origen.

1.9.2. Decoder

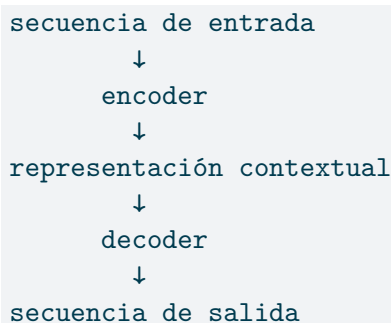
El decoder genera una secuencia de salida.

Durante esa generación utiliza:

- los tokens ya generados;
- las representaciones producidas por el encoder;
- una máscara causal que impide acceder a posiciones futuras.

En un sistema de traducción, el decoder produciría progresivamente la frase en el idioma de destino.

De forma simplificada:



1.10. Variantes de la arquitectura

No todos los modelos basados en Transformers utilizan simultáneamente encoder y decoder.

A partir de la arquitectura original aparecieron varias familias.

1.10.1. Modelos de tipo encoder

Utilizan principalmente bloques encoder.

Están orientados a construir representaciones del texto y resultan adecuados para tareas como:

- clasificación;
- extracción de información;
- análisis de contenido;
- comparación semántica;
- comprensión de documentos.

1.10.2. Modelos de tipo decoder

Utilizan bloques decoder con atención causal.

Generan el texto progresivamente, prediciendo cada token a partir de los anteriores.

Son la base de muchos modelos generativos y asistentes conversacionales.

1.10.3. Modelos encoder-decoder

Mantienen ambos componentes.

Resultan especialmente adecuados cuando se transforma una secuencia de entrada en otra secuencia de salida:

- traducción;
- resumen;
- reescritura;
- conversión entre formatos;
- generación condicionada por un documento.

Estas categorías comparten los principios fundamentales del Transformer, pero organizan sus bloques de manera diferente según el objetivo.

1.11. Procesamiento en paralelo

Una ventaja importante del Transformer es que, durante el entrenamiento, puede procesar simultáneamente múltiples posiciones de una secuencia.

En una arquitectura estrictamente recurrente, el cálculo de una posición depende del estado producido en la posición anterior.

En un Transformer, las relaciones entre tokens pueden calcularse mediante operaciones matriciales paralelas.

Esto permite aprovechar de forma eficiente hardware especializado, como las unidades de procesamiento gráfico.

La capacidad de paralelización contribuyó a entrenar modelos con:

- mayores volúmenes de datos;
- más parámetros;
- más capas;
- vocabularios amplios;
- contextos progresivamente mayores.

El Transformer no es importante únicamente por representar mejor ciertas relaciones lingüísticas. También hizo viable escalar el entrenamiento de modelos de lenguaje.

1.12. Capas y representaciones

Las representaciones no permanecen iguales mientras atraviesan el Transformer.

En las primeras capas pueden aparecer relaciones relativamente locales o estructurales.

En capas posteriores, las representaciones pueden integrar información procedente de partes más amplias del contexto.

No debemos imaginar, sin embargo, una jerarquía perfectamente organizada en la que cada capa tenga una función única y fácilmente interpretable.

El conocimiento y el comportamiento del modelo están distribuidos entre:

- numerosas capas;
- múltiples cabezas de atención;
- redes feed-forward;
- embeddings;
- conexiones internas;
- millones o miles de millones de parámetros.

Las capacidades del modelo emergen de la interacción conjunta de estos componentes.

1.13. Transformer y ventana de contexto

La atención se calcula sobre los tokens disponibles dentro de la ventana de contexto.

Cuanto mayor sea el contexto, mayor puede ser el número de relaciones que deben evaluarse.

En la formulación clásica, el coste de la autoatención crece rápidamente al aumentar la longitud de la secuencia, porque cada posición puede compararse con muchas otras posiciones.

Por esta razón, procesar contextos muy extensos requiere una cantidad considerable de memoria y capacidad de cálculo.

Las arquitecturas posteriores han introducido distintas optimizaciones para:

- reducir el coste de la atención;
- trabajar con secuencias más largas;
- reutilizar cálculos anteriores;
- seleccionar relaciones relevantes;
- distribuir el procesamiento.

La ventana de contexto no es, por tanto, un límite puramente arbitrario. Está relacionada también con el coste técnico de procesar las relaciones entre tokens.

1.14. El Transformer no es un LLM

Transformer y LLM tampoco son conceptos equivalentes.

El Transformer es una **arquitectura**.

Un LLM es un modelo de lenguaje de gran escala que normalmente utiliza una arquitectura basada en Transformers y ha sido entrenado con grandes cantidades de datos.

Podemos expresarlo así:

Transformer = arquitectura

modelo de lenguaje = modelo entrenado para trabajar con lenguaje

LLM = modelo de lenguaje de gran escala

Un Transformer puede utilizarse para tareas que no son estrictamente lingüísticas.

Existen modelos basados en esta arquitectura para trabajar con:

- imágenes;
- audio;
- vídeo;
- datos biológicos;
- series temporales;
- sistemas multimodales.

La arquitectura define cómo se procesa y relaciona la información. El entrenamiento y los datos determinan qué capacidad concreta desarrolla el modelo.

1.15. Una arquitectura dentro de un sistema inteligente

Un modelo basado en Transformers continúa siendo solo un componente.

Para convertirlo en un sistema inteligente útil, sigue siendo necesario decidir:

- qué información recibe;
- cómo se construye el contexto;
- qué modelo se utiliza;
- qué resultados son válidos;
- cómo se verifican;
- qué herramientas puede emplear;
- qué acciones puede ejecutar;
- qué controles se aplican;
- quién asume la responsabilidad.

La arquitectura aporta capacidad para representar, relacionar y generar información.

El sistema inteligente determina cómo se utiliza esa capacidad.

1.16. Del Transformer al modelo de lenguaje

El recorrido conceptual queda ahora así:

texto → tokens → embeddings → información posicional → atención multicabeza
→ bloques Transformer → representaciones contextuales

Todavía falta explicar cómo se utiliza esta arquitectura para aprender el lenguaje.

Un Transformer no nace sabiendo escribir, resumir, traducir o responder preguntas. Estas capacidades aparecen como resultado de un proceso de entrenamiento sobre grandes cantidades de secuencias.

En la siguiente sección estudiaremos qué es un **modelo de lenguaje**, qué significa predecir el siguiente token y cómo esa tarea aparentemente sencilla puede dar lugar a capacidades mucho más amplias.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. «Attention Is All You Need». *Advances in Neural Information Processing Systems* 30.